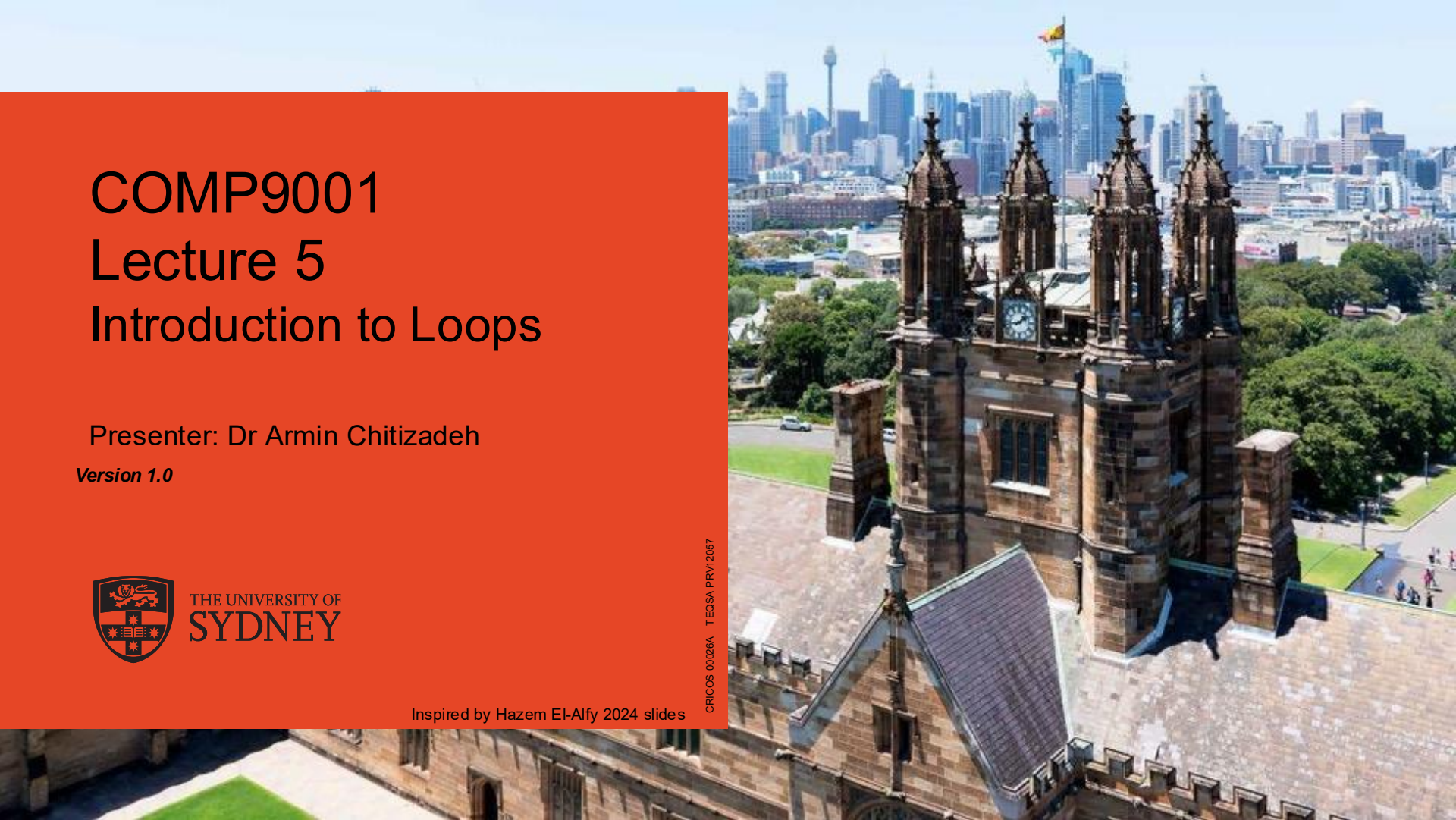
COMP9001
Lecture 5
Introduction to Loops

Presenter: Dr Armin Chitizadeh

Version 1.0



THE UNIVERSITY OF SYDNEY

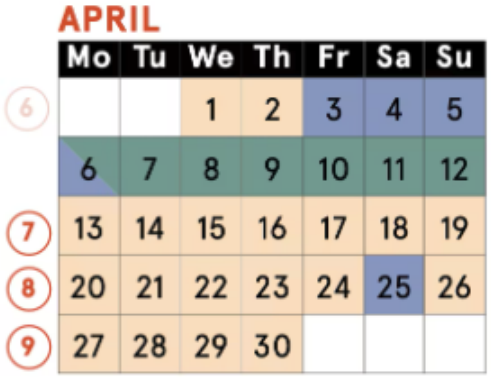
Inspired by Hazem El-Alfy 2024 slides

CRICOS 00026A TEQSA PRV12057

We recognise and pay respect to the Elders and communities – past, present, and emerging – of the lands that the University of Sydney's campuses stand on. For thousands of years they have shared and exchanged knowledges across innumerable generations for the benefit of all.

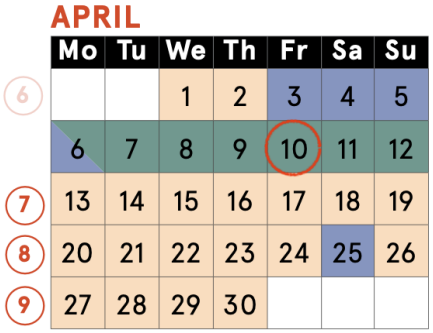
THE UNIVERSITY OF SYDNEY

What is your plan for mid-semester break?



https://www.menti.com/al7gwbeuqxgq

Mid-semester Review Session



Name	Description	Allocated Location Description	Planned Size	Scheduled Days	Scheduled Start Time	Scheduled End Time	Duration	Activity Dates (Individual)
COMP9001-S1C-NE-CC/Assessment-IGN/02	Mid semester break Review Session	H70.02.2100.Belinda Hutchinson Building.BHB Learning Studio 2100	60	Friday	TBA	TBA	Approximately 6 to 7 hours but you can join in and leave as you wish	10/04/2026

More is better, if it is clear

“More is better, if it is clear”

Armin Chitizadeh

Three wishes from Jeannie!

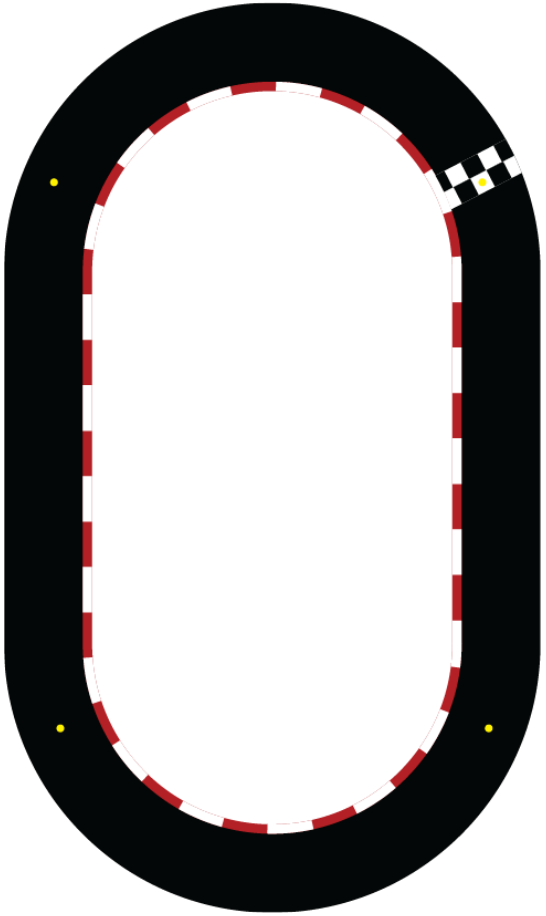
Can We Program Jeannie Better?

You can make three wishes

Sphinx (the kind one)

“Answer me:
What walks on four legs in the morning,
two in the afternoon, and
three in the evening”

The kind of keeps asking the question until it hears the correct answer



Iterations in Life

- Life is full of repetitive actions.
  - Make cups of tea until everyone present has a cup
    - Prepare cup
    - Pour tea in cup
    - Serve
  - Shopping – shop until everyone on list have a gift.
    - Decide gift.
    - Search for item online or in-shop.
    - Purchase item.
    - Wrap item.
  - Practice makes perfect.
    - Gaming: Repeat level until [level is cleared/item is found/top of chart]
    - Studying: Attempt programming exercises until I [master programming/get tired/give up/have eureka moment]

- Do you notice anything?

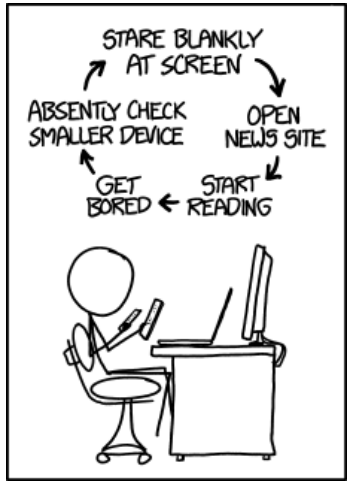
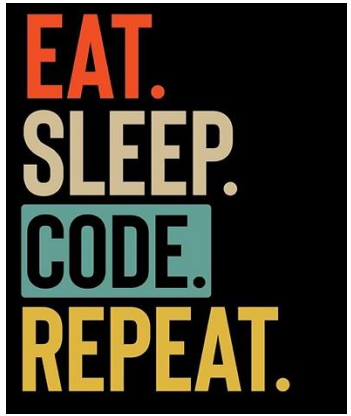


Image credit: https://xkcd.com/1411/



Types of Loops

- Loops enables us to set up *arbitrary patterns* of repetition: do something once, a million times, or until a condition is no longer true.

- Two Types:

  - **Count-controlled** Loops
    - Repeat a set of actions a <u>known</u> number of times.
    - Situation of use:
      - Operating on everything in a set
      - Scanning through a list of command-line arguments

  - **Event-controlled** Loops
    - Repeat a set of actions an <u>unknown</u> number of times until the sentinel value (unlikely data) is entered or flag value (False) is encountered, or some limit reached.
    - Situation of use:
      - Read a file until all end of file

Count-controlled Loops

Loops IRL

- A race requires participants to run around the track **FIVE(5)** times.

- How do you track the number of times a participant had run around the track?

- If you are a participant in the race, when do you stop running around the track?

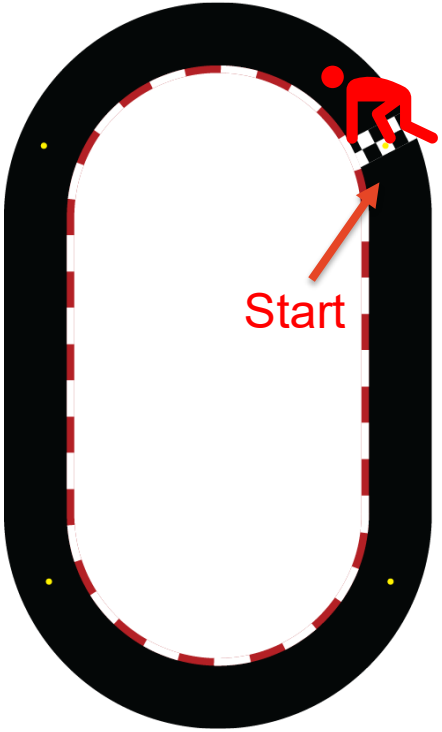
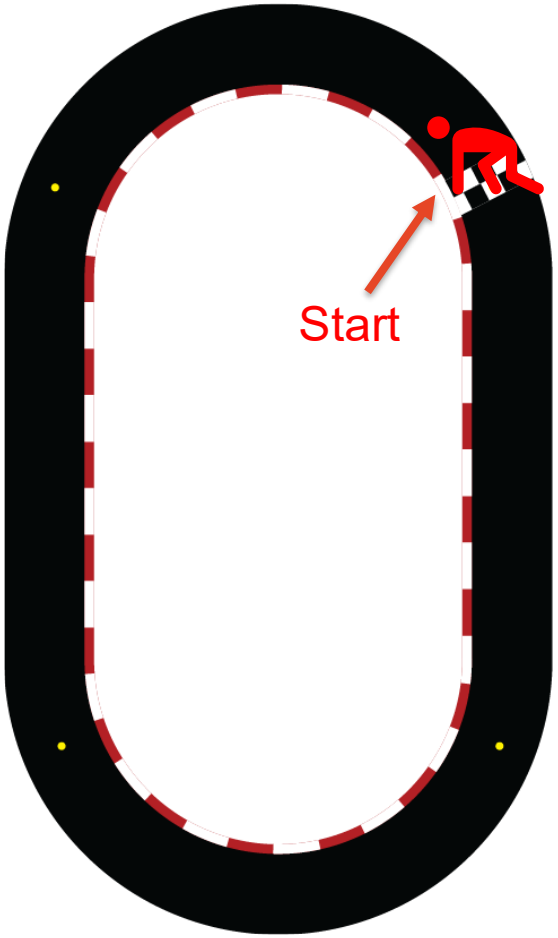


Image credit: Clipartmax - Race Clipart Flag Png

Anatomy of Loops

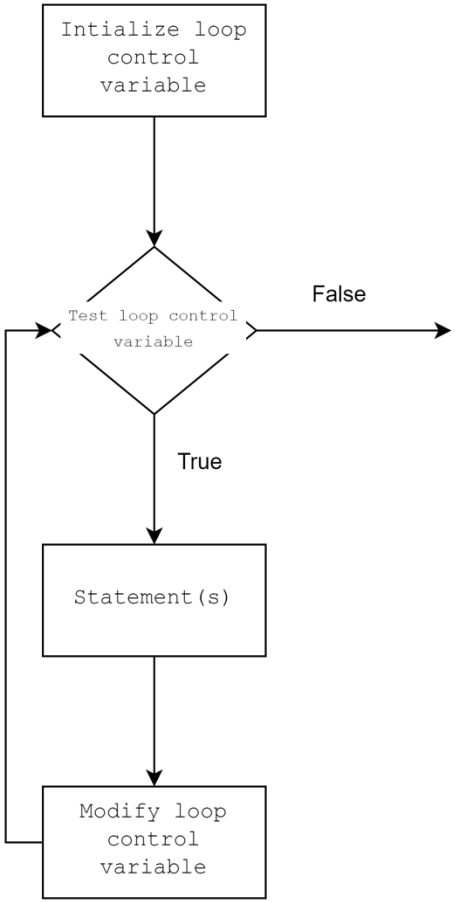


round = 1

round <= 5

RUN!!!

round += 1



```
i = i + 1          is same as          i += 1

i = i - 1          is same as          i -= 1
```

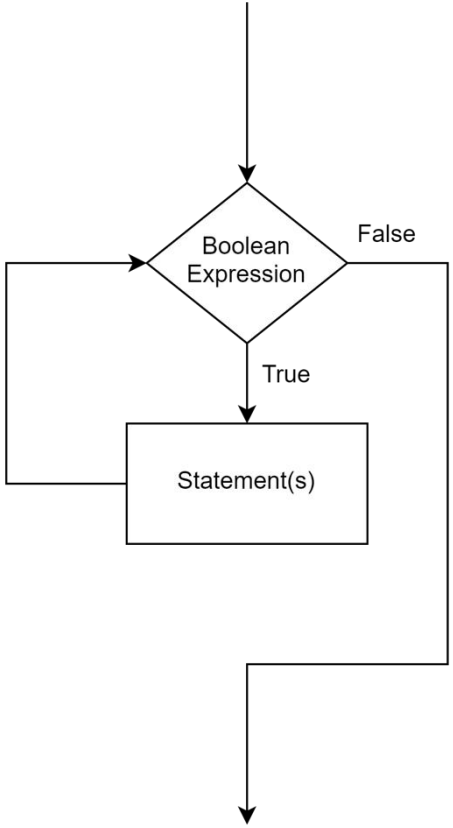
Control Flow: `while`

Control Flow: `while`

- Syntax:

```
while <boolean expression>:
    Statement(s)
```

- Flowchart

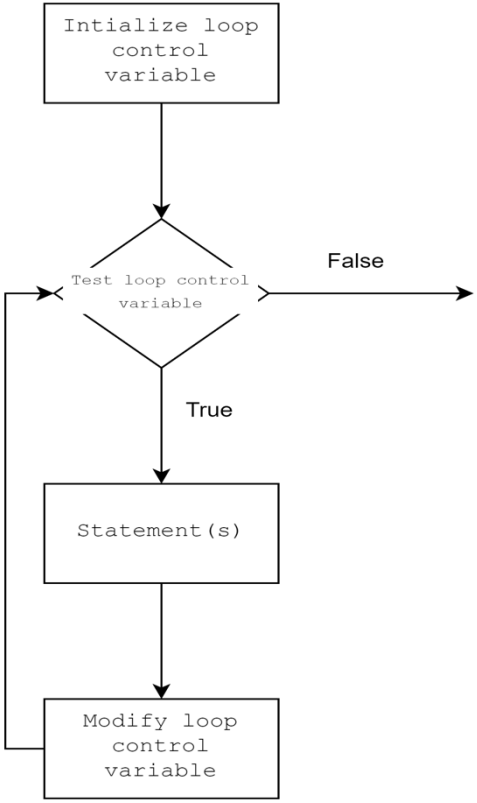


Example: Simple Loop

- Program:

```
1 i = 1
2 while i <= 5:
3 ····print(f"Round {i}: RUN!!!")
4 ····i = i + 1
5 print(f"Round {i}: Stop!")
```

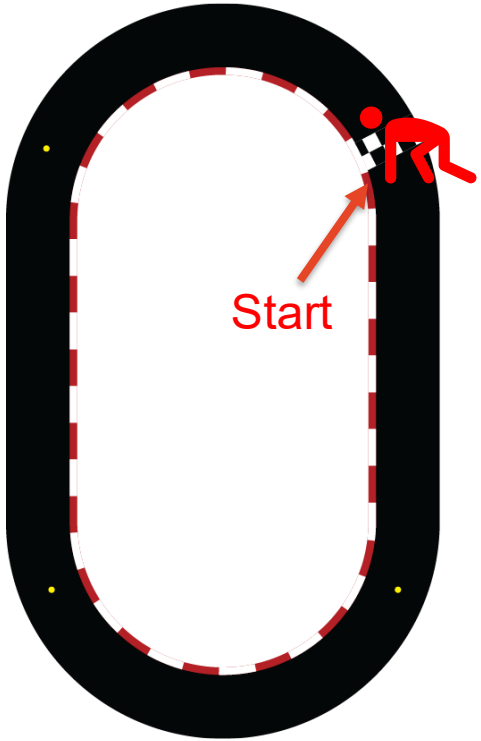
- Flowchart:

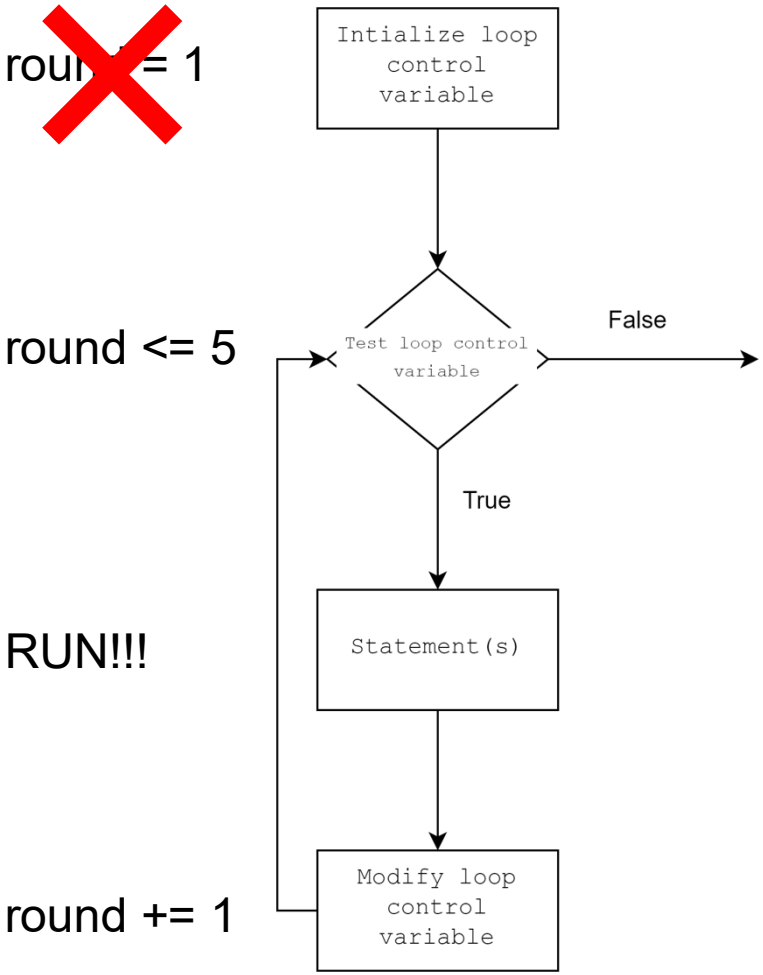


Activity:

- Go to: Ed Lesson (Simple Loop)
- Run the program `demo.py`

Anatomy of Loops





What Happens If the Components are missing?

What Happens If the Components are missing?

- Initialize loop control variable
  - Unable to start loop.

- Condition to test loop control variable condition
  - Syntax error
  - Logic error – loop won't start or wrong number of loops

- Modification of loop control variable
  - Logic error – wrong number of loops
  - Worst-case scenario : infinite loops

Infinite Loops

- A loop that *never* exits is called an infinite loops
- This happens because the condition ***is*** **always True**.

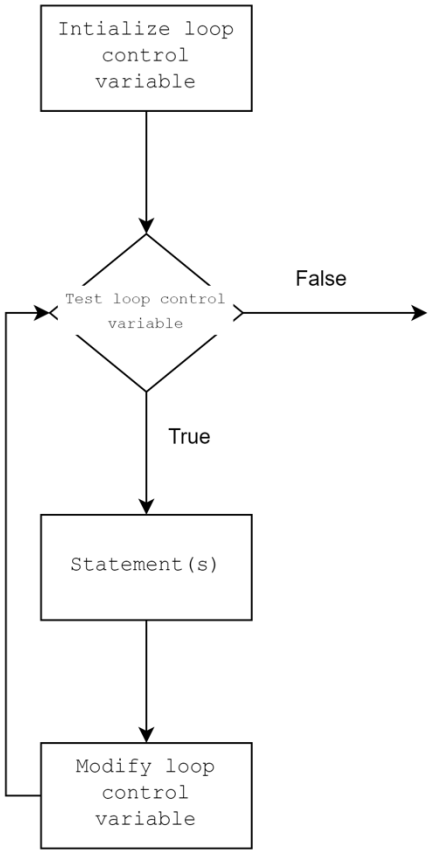
- If an infinite loop is unintentional, you ***didn't*** design this, you can issue a Keyboard Interrupt command <CTRL-C> to stop execution.

- Tips: Prevent accidental infinite loops by ensuring that the body contains:
    - Statements to modify the loop control variable OR
    - Statements to terminate infinite loop when event is achieved

Number of Iterations

- Many ways to implement the same number of known iterations.

Initialize	Test Condition	Modify	Total Iterations
`i = 0`	`i < 5`	`i += 1`	5
`i = 0`	`i <= 4`	`i += 1`	5
`i = 1`	`i < 6`	`i += 1`	5
`i = 1`	`i <= 5`	`i += 1`	5



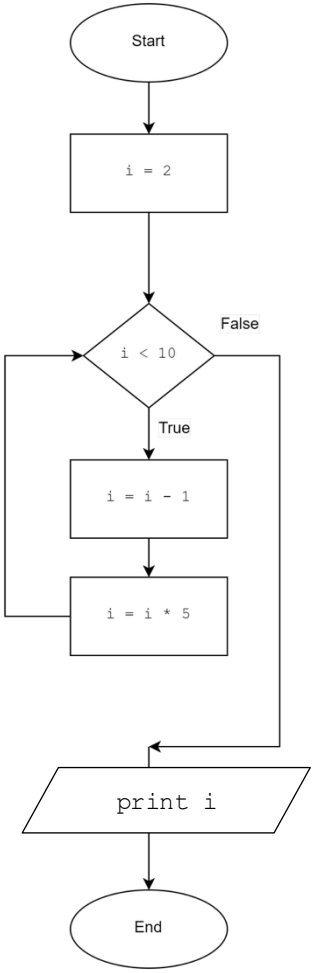
Practice: Flowchart

- What is the final value of `i`?
- How many iteration will there be in total?

- Complete informal trace:

`i`	`i < 10`	`i = i - 1`	`i = i * 5`
2	True		
5			

-



`i`	`i < 10`	`i = i - 1`	`i = i * 5`
2	True	1	5
5	True	4	20
20	False		

Create the Jeannie Program

(Repeat a set of actions)

Remember how we wrote the RUNN Program?

```
i = 1
while i <= 5:
    print(f"Round {i}: RUN!!!")
    i = i + 1
print(f"Round {i}: Stop!")
```

Link to ed: Jeannie

Applications of Loops

- Repeat set of actions
  - RUN!!! Example
  - Jeanie Example
  - Must have a counter variable that is incremented with each iteration of a loop

- Summation – calculate running total of a set of values
  - Calculate average of 3 numbers.
  - Must have a variable to hold the sum in addition to loop control variable

- Iteration through elements in a collection
  - Display out characters in a string
  - Must have a counter variable that is incremented with each iteration of a loop

- Count occurrences – find and tally occurrences
  - Count total occurrences ‘a’ in a word
  - Must have variables to hold each count ***in addition*** to loop control variable

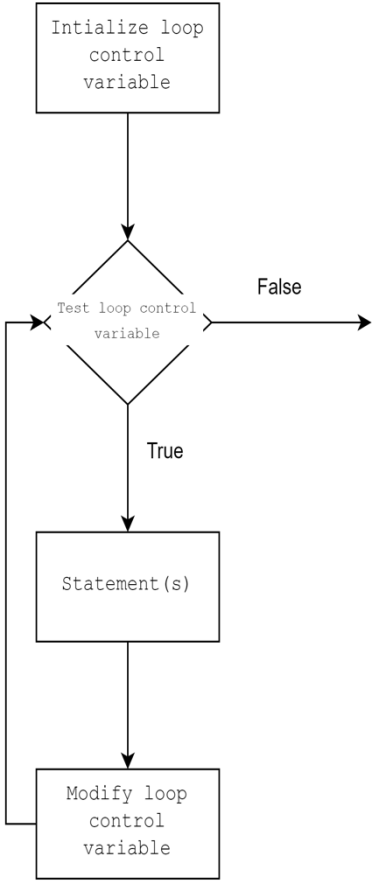
How to design loops?

For a given problem, determine:

1. Type of loop:
   - count-controlled loop (known number)?
   - event-controlled loop (unknown number)?

2. Condition to end iteration:
   - Known: Number of iterations?
   - Unknown:
     - Sentinel value (if any)?
     - When to raise flag?

3. Body of loop contents:
   - Sequence of actions to be executed during each iteration?
   - Statement to modify loop control variable?



Application of loops - Summation

- Write a program to compute the average of three integers given by users

```
$ python3 average.py
Number 1: 1
Number 2: 2
Number 3: 3
Average: 2.0
```

```
$ python3 average.py
Number 1: 5
Number 2: 10
Number 3: 15
Average: 10.0
```

- Questions:
  - Are the number of iterations **known** or **unknown** to you?
  - What is the **first number** at the start of the loop?
  - What is the **final number** at the end of the loop?
  - List the repeated actions to be completed ***inside*** the loop.
  - List the actions to be completed ***after*** the loop.

Summation - Initial Value

- Write a program to compute the average of three integers given by the user.

```
$ python3 average.py
Number 1: 1
Number 2: 2
Number 3: 3
Average: 2.0
```

```
$ python3 average.py
Number 1: 5
Number 2: 10
Number 3: 15
Average: 10.0
```

- Think about your actions IRL:
  - For this problem, if you are playing the role of the computer IRL and the output is written by you onto a paper, what is the first number at the start of the loop? Number of iterations

Summation – Termination Condition

- Write a program to compute the average of three integers given by the user.

```
$ python3 average.py
Number 1: 1
Number 2: 2
Number 3: 3
Average: 2.0
```

```
$ python3 average.py
Number 1: 5
Number 2: 10
Number 3: 15
Average: 10.0
```

- Think about your actions IRL:
  - For this problem, if you are asking another person for THREE numbers, when do you stop asking for another number?

Summation – Repeated Actions

- Write a program to compute the average of three integers given by the user.

```
$ python3 average.py
Number 1: 1
Number 2: 2
Number 3: 3
Average: 2.0
```

```
$ python3 average.py
Number 1: 5
Number 2: 10
Number 3: 15
Average: 10.0
```

- Think about your actions IRL:
  - How do you find the sum of three numbers?
  - List the actions you take to calculate the average of three numbers given to you verbally by another person.
  - List the actions to be taken by the computer to calculate the average of three numbers.

Summation – Programming

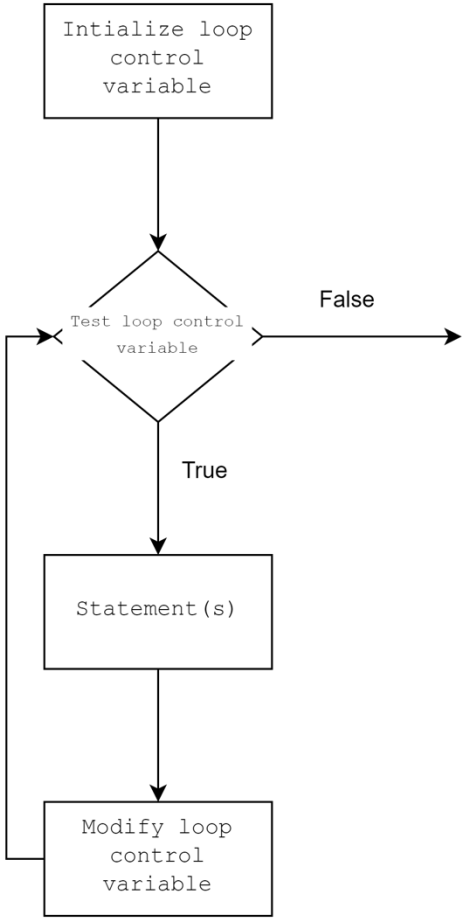
- What we know so far?
  - Get input from users using `input([prompt])`
  - Values must be stored to memory before they can be re-use. `variable = value`
  - Perform mathematical calculation on integers and floats: `+`, `/`
  - Compare numbers using Boolean operators: `<=`, `<`
  - Display output to the terminal using `print()`

- Can you identify the actions that goes into the body of the loop?

- Can you identify the actions that goes after the body of the loop?

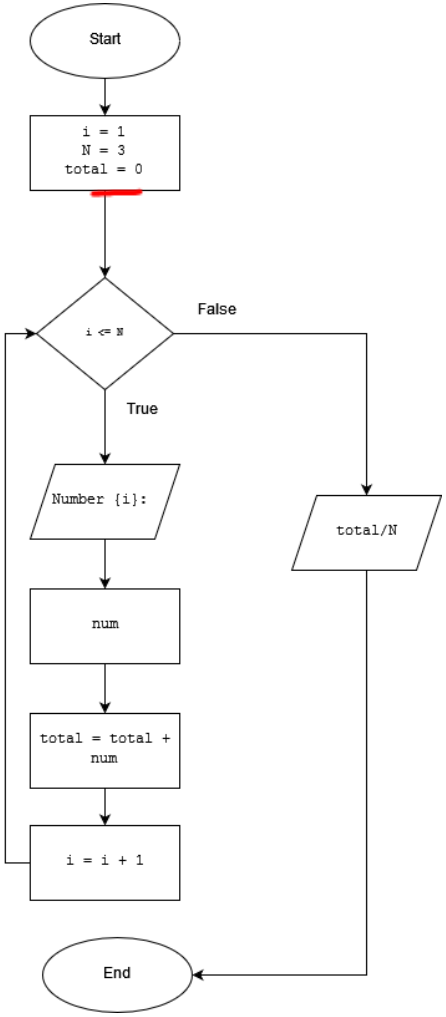
Application of Loops - Summation

- Type of loop: count-controlled loop.
- Total Iterations: 3
- Algorithm:
    - Set initial value for loop control variable.
    - Initialize variable to hold total as 0.
    - While statement to compare loop control variable with terminating condition.
        - Get input and save value.
        - Add to running total.
        - Modify loop control variable to track iteration.
    - Divide the total by 3
    - Display result.



Application of Loops - Summation

- Type of loop: **count-controlled** loop.
- Total Iterations: 3
- Algorithm:
  - Set initial value for loop control variable.
  - Initialize variable to hold total as 0.
  - While statement to compare loop control variable with terminating condition.
    - Get input and save value.
    - Add to running total.
    - Modify loop control variable to track iteration.
  - Divide the total by 3
  - Display result.
- Write the program: Ed Lesson (Application – Summation)



Break!

https://www.youtube.com/watch?v=CSvFpBOe8eY

System Of A Down - Chop Suey! (Official HD Video)

Applications of Loops

- Repeat set of actions
  - Jeanie !!! Example
  - Must have a counter variable that is incremented with each iteration of a loop.

- Summation – calculate running total of a set of values
  - Calculate average of 3 numbers.
  - Must have a variable to hold the sum *in addition* to loop control variable.

- Iterating through elements in a collection
  - Display out characters in a string.
  - Must have a counter variable that is incremented with each iteration of a loop.

- Count occurrences – find and tally occurrences.
  - Count total occurrences ‘a’ in a word
  - Must have variable to hold each count *in addition* to loop control variable.

Control Flow: `for`

Control Flow: `for`

- Syntax:

```
for element in sequence:
    Statement(s)
```

Printing Class Roll

```
class_roll = ["Joshua", "Ehab", "Michael", "Daniel", "Tim", "Nicole"]
```

(1)

```
for name in class_roll:
    print(f"Hello {name}")
```

(3)

```
i = 0
while i < len(class_roll):
    print(f"Hello {class_roll[i]}")
    i = i + 1
```

(2)

```
for i in range(0, len(class_roll)):
    print(f"Hello {class_roll[i]}")
```

Iterating through elements

- Recall from W2: A `string` is actually a sequence of characters.
- We can access each character (element) by referring to its index. This rule applies to other collections e.g. lists, tuples, numpy arrays.
- Python follows zero-based numbering. The first element always starts with 0. How would a zero-based numbering system affect your loop design?
- We can use loops to iterate through the elements.
- Go to Ed Lesson (Iterating Through Elements) and run the codes.

Your task

- Can you spot the line that:
  - Initializes the loop control variable?
  - Loop terminating condition?
  - Modifies the loop control variable?
- Draw the flowchart for both programs.

Extra in Python: From `while` loop to `for` loop

- Now, can you spot the similarities between both loops?

```python
i = 1
while i <= 5:
    print(f"Round {i}: RUN!!!")
    i = i + 1
print(f"Round {i}: Stop!")
```

```python
for i in range(1, 6):
    print(f"Round {i}: RUN!!!")
print(f"Round {i+1}: Stop!")
```

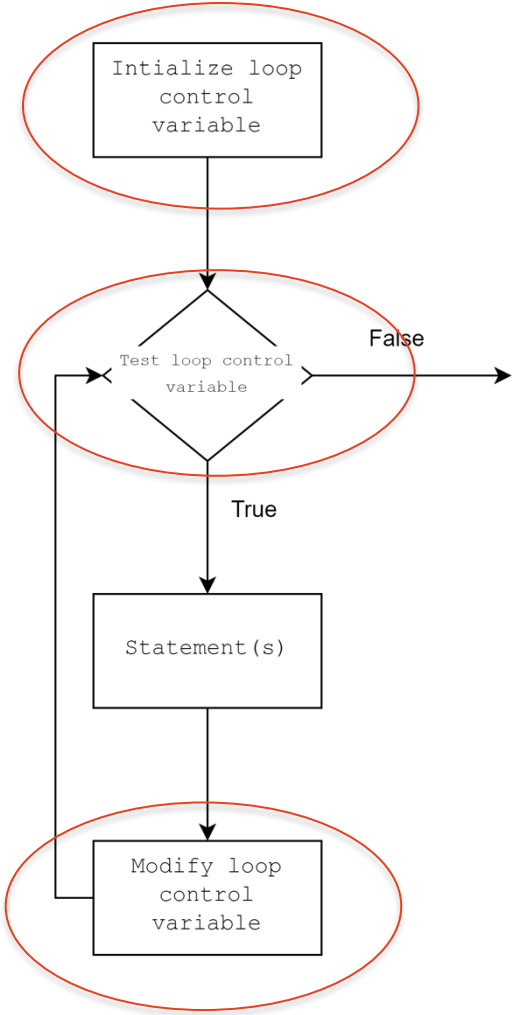
- Can you identify these in the `for` loop?
    - Loop control variable
    - Terminating condition
    - Modification of control variable

We will talk more about `for` loops in Week 9.

Summary for initial part

- Two types of loops:
  - Count-controlled loops (known)
  - Event loops (unknown)

- Count-controlled loops:
  - Many ways to implement same number of iterations
  - Applications of loops



Event Controlled Loops

Event Controlled Loops

“Answer me:
What walks on four legs in the morning,
two in the afternoon, and
three in the evening”
Won’t stop until you answer correctly

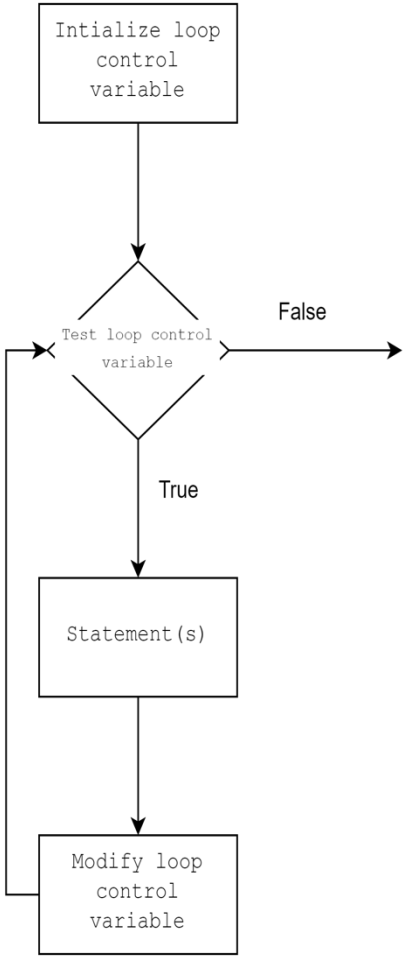
Recap: How to design loops?

For given problem, determine:

1. Type of loop:
   - count-controlled loop (known number)?
   - event-controlled loop (unknown number)?

2. Condition to end iteration:
   - Known: Number of iterations?
   - Unknown:
     - Sentinel value (if any)?
     - When to raise flag?

3. Body of loop contents:
   - Sequence of actions to be executed during each iteration?
   - Statement to modify loop control variable?

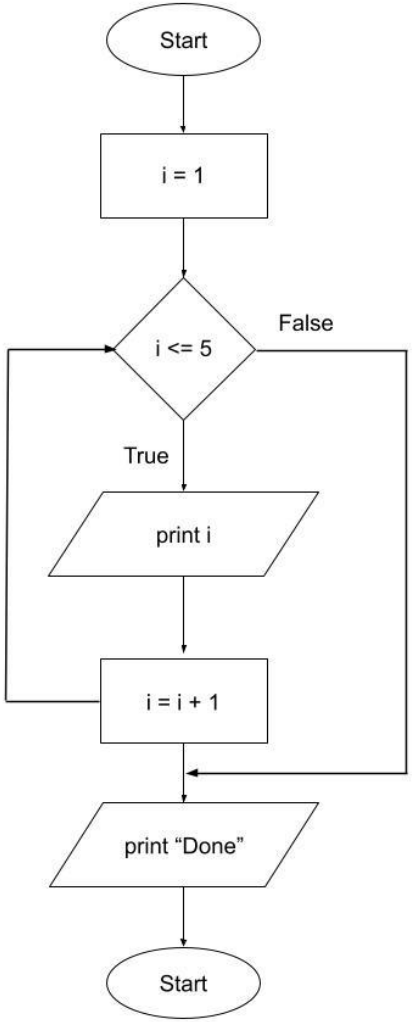


Recap: Flowchart

- How many iterations do you expect?
- What will be the output?

i	i <= 5	i = i + 1
1	True	

- Check your answer: Ed Lesson (Practice 2:Flowchart).



Recap: Infinite Loops

- A loop that *never* exits is called an infinite loop.

- This happens because the condition *is* always True.

- If an infinite loop is unintentional, you ***didn't*** design this, you can issue a Keyboard Interrupt command <CTRL-C> to stop execution.

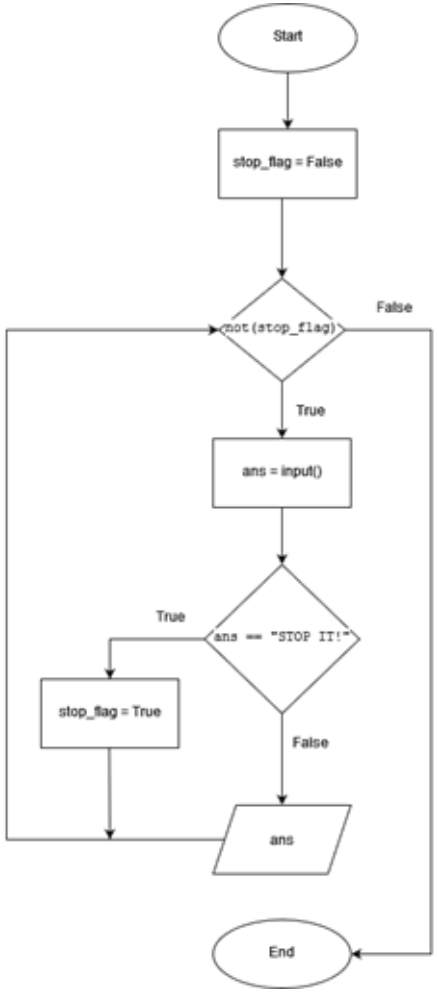
- Tips: Prevent accidental infinite loops by ensuring that the body contains:
    - Statements to modify the loop control variable OR
    - Statements to terminate infinite loop when event is achieved

Event Controlled Loops

- Loop terminates when an event happens:

  - Sentinel value, unlikely value that the data can take, is received.
  - Flag that controls the loop changes to **`False`**.

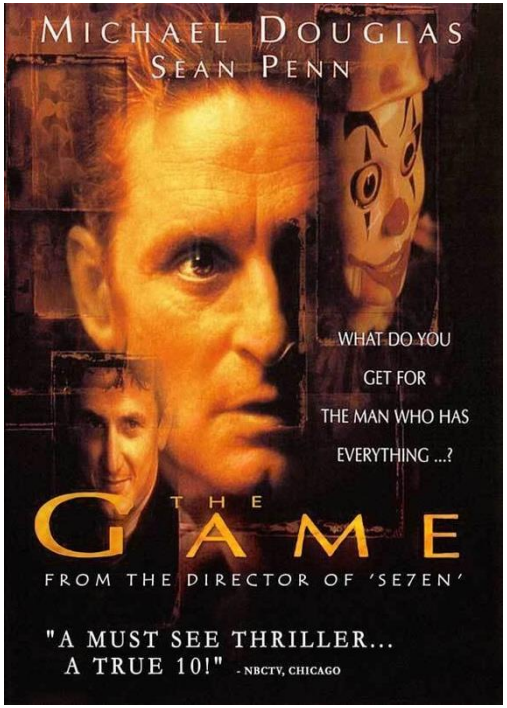
- Can you spot?
  - Loop control variable?
  - Terminating condition?
  - Modification of the control variable?



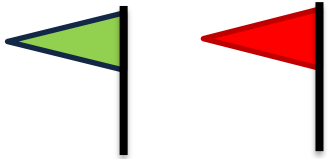
Event Control Loop

Sentinel value

- Like “Safe-Word” in Extreme Haunted Houses



Flag

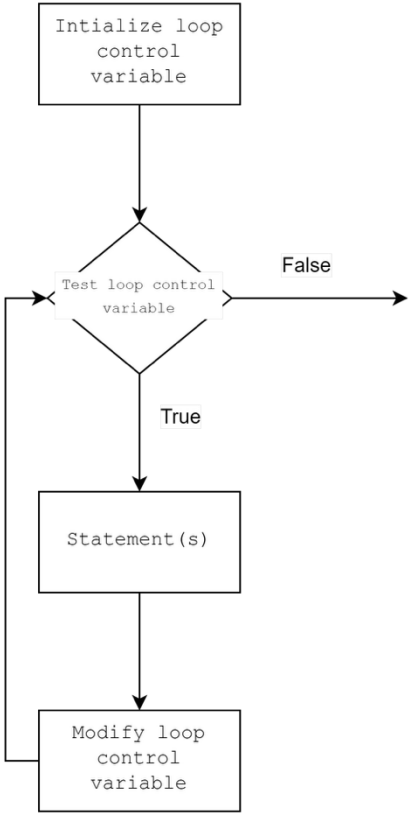


- Use flag in to make your code clearer, if it makes it clearer

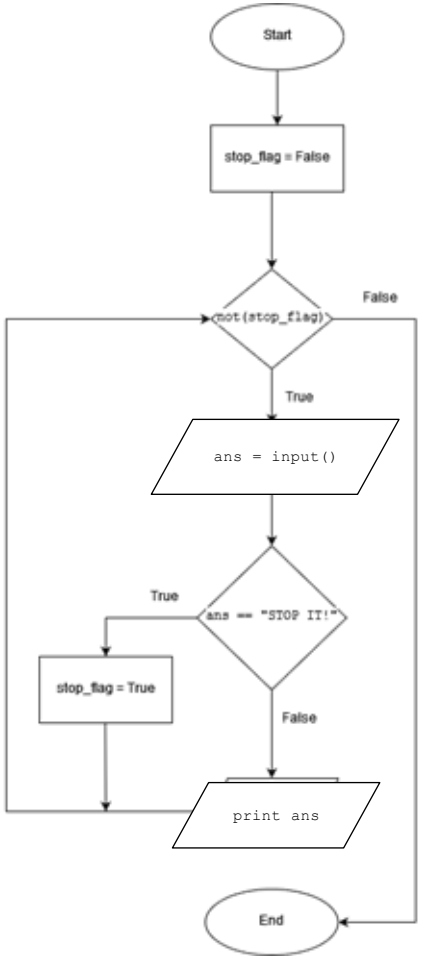
Anatomy of Loops

Activity: Can you spot the similarities between both flowcharts?

count-controlled loop



event-controlled loop

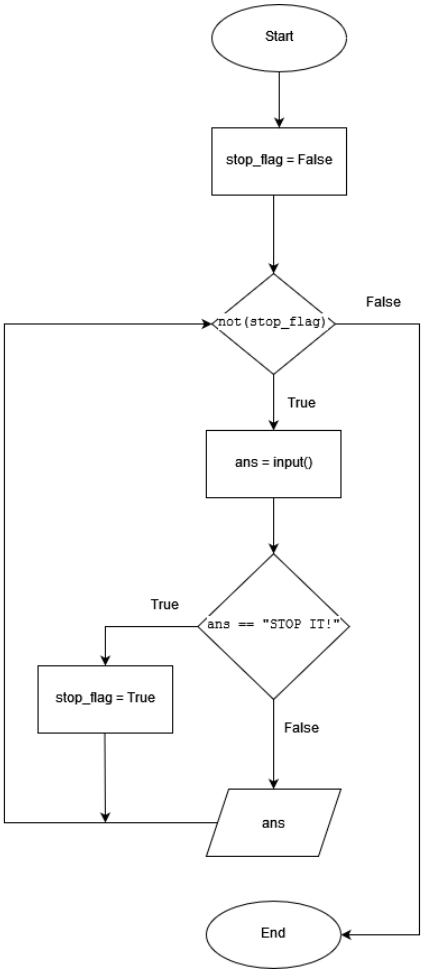


Programming Basics

- Functions (W6)
  - Associate a set of actions to a variable (function name)
- Conditionals (W3)
  - Make decisions
    - Single branch – **`if`**
    - Alternate branch – **`else`**
    - Multiple options – **`elif`**
- Loops (W4)
  - Repeat a set of actions: count-controlled/event-controlled
    - **`while`** loops – count-controlled and event-controlled
    - **`for`** loops [To be covered in W9]
      - Simplifies count-controlled loops
      - Variations depending on programming languages

Echo Again – Flowchart to Codes

- Go to Ed Lesson.
- Review the codes.
- What should the user input to stop the program from echoing?
- Run the program and check your answer.



Echo Again 2.0

- Modify the `echo.py` program to spell each letter in a given word. Stop when the user enters "STOP IT!"

```
$ python3 echo.py
#STOP IT!
```

```
$ python3 echo.py
#hello
h
e
l
l
o
#help
h
e
l
p
#stop
s
t
o
p
#STOP IT!
```

Echo Again 2.0 – Loop Design

- Answer these questions:

1. Type of loop:
   - count-controlled loop (known number)?
   - event-controlled loop (unknown number)?

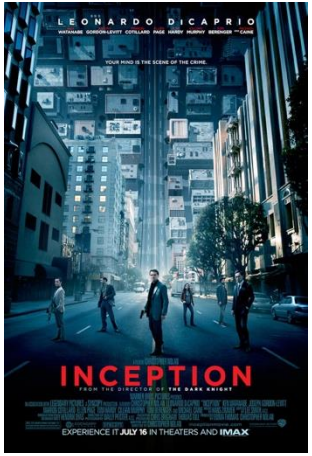
2. Condition to end iteration:
   - Known: Number of iterations?
   - Unknown:
     - Sentinel value (if any)?
     - When to raise flag?

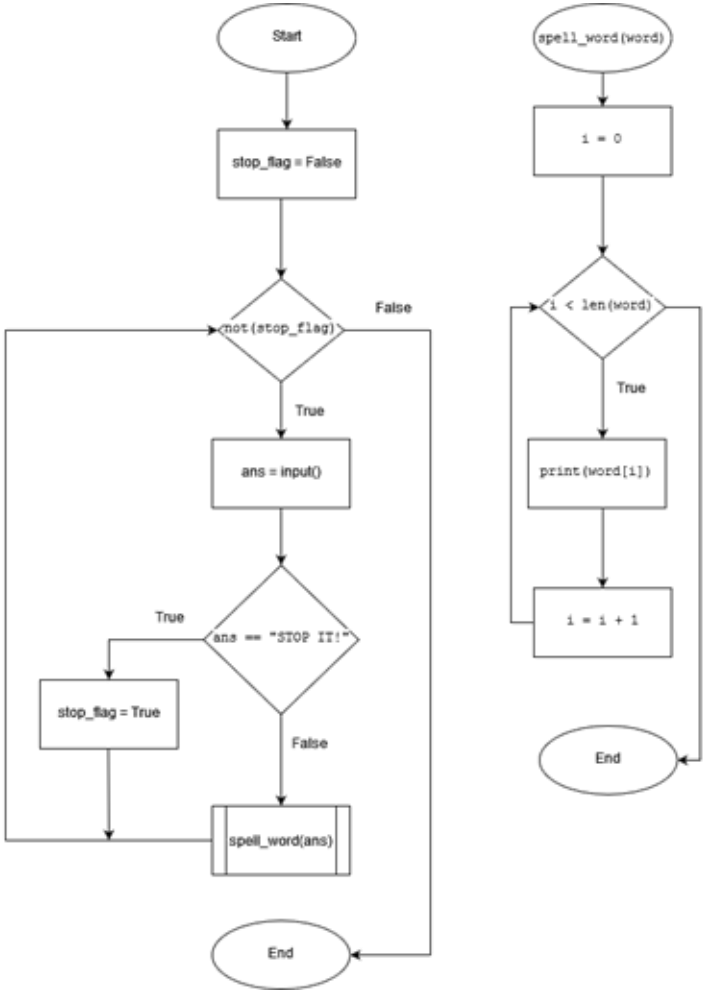
3. Body of loop contents:
   - Sequence of actions to be executed during each iteration?
   - Statement to modify loop control variable?

Echo Again 2.0 - Flowchart

Your Task:

- Implement the flowchart in [Ed Lesson Echo Again 2.0]

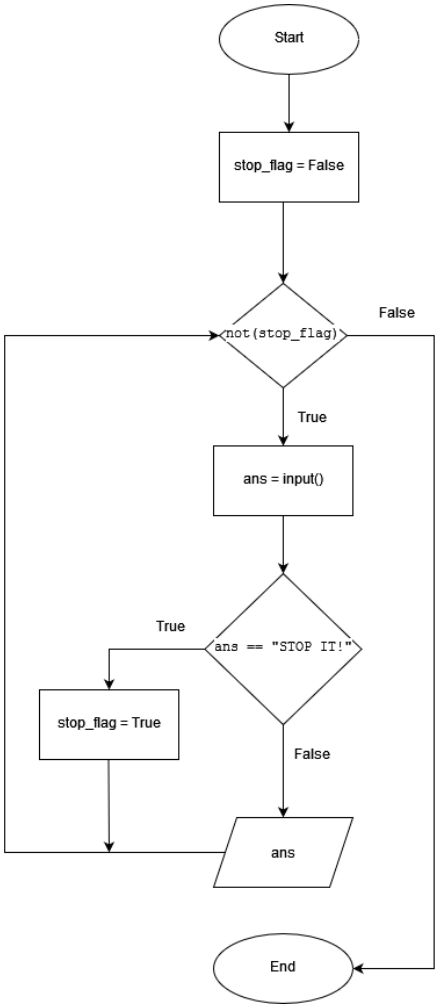




Summary

- Event controlled loops:
  - Initialize loop control variable – flag.
  - Check for event inside the loop body.
  - Modify loop control variable – change flag to `False`.

- Programs can contain a combination of variables, functions, expressions, conditionals and/or loops.



Reading This Week

- **Chapter 7.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.

- **Chapter 5.** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.

Reading This Week

- **Chapter 7.** Downey, A. B. (2015). Think Python: How to Think Like a Computer Scientist (2e ed.). O'Reilly Media, Incorporated.

- **Chapter 5.** Liang, Y. (2022). Introduction to Python Programming and Data Structures, Global Edition. Pearson Education Limited.

Fully anonymous feedback form for COMP9001 students – Fill out form

Updates